# DM 1 - Vérification du statut "de recherche" pour un arbre binaire

Dans ce DM, on se propose d'implémenter différentes méthodes permettant de vérifier si un arbre binaire est un arbre binaire de recherche (ABR) et de comparer leur efficacité expérimentale.

Dans tout ce DM, on considérera le type suivant d'arbre binaire :

```c
typedef struct noeud {
    int valeur;
    struct noeud *fg, *fd;
} Noeud, * Arbre;
```

Pour simplifier, on ne considérera que des arbres binaires **sans doublon**.

L'évaluation du code rendu prendra particulièrement en compte la documentation du code, la libération de la mémoire ainsi que les tests réalisés du code.

Enfin, lors de la création de statistique permettant de comparer différents algorithmes, on utilisera le type `long long` pour les compteurs afin de limiter les dépassements de capacité lors des tests sur de très grands arbres ou lors de l'accumulation de mesures sur plusieurs répétitions.

## 1 Méthodes de vérification

Toutes les fonctions demandées dans cette partie seront présentés dans un fichier `est_ABR.c`. Le fichier `est_ABR.h` associé sera alors crée pour pouvoir les utiliser dans un autre fichier.

Lorsqu'une modification de fonction sera demandée (ou réalisée), une sauvegarde de la fonction sera toujours présente dans le fichier `est_ABR_V1.c`.

### 1.1 Application de la définition - Approche naïve

L'approche naïve consiste à reprendre ce que vous avez déjà fait en TP, c'est-à-dire vérifier récursivement, pour chaque nœud $n$, que toutes les valeurs de son sous-arbre gauche sont strictement inférieures à l'étiquette de la racine, et que toutes celles de son sous-arbre droit lui sont strictement supérieures. Pour cela, on pourra utiliser/modifier les fonctions `int abr_min(Arbre a)` et `int abr_max(Arbre a)` écrite en TP. Pour mémoire, elles renvoient respectivement la plus petite étiquette et la plus grande étiquette de l'arbre non vide `a`.

1. Écrire dans un premier temps la fonction `int est_abr_naif(Arbre a)` qui renvoie 1 si l'arbre binaire `a` est bien un arbre binaire de recherche, 0 sinon.
2. Modifier dans un second temps les fonctions précédentes pour obtenir une fonction `int est_abr_naif(Arbre a, long long * nb_visites)` qui renvoie 1 si l'arbre binaire `a` est bien un arbre binaire de recherche, 0 sinon. Celle-ci déterminera en plus le nombre de nœuds visités pour déterminer la valeur de retour.

### 1.2 Application de la définition - Approche optimisée

Cette méthode consiste à modifier la méthode précédante en ne parcourant l'arbre qu'une seule fois. L'idée est de propager à travers les appels récursifs les valeurs minimales et maximales observées pour chaque sous-arbre (donc, sans avoir à reparcourir les sous-arbres comme dans la version naïve précédente).

1. Écrire la fonction `est_abr_definition_aux(Arbre a, int *min, int *max)` qui valide le statut de recherche de l'arbre binaire `a` en renvoyant 1 ou 0 selon, et qui mets à jour les valeurs pointées par `min` et `max` uniquement lorsque l'arbre `a` est un arbre binaire de recherche.
2. Écrire la fonction `est_abr_definition(Arbre a)` qui valide alors le statut de recherche de l'arbre binaire `a`.
3. Modifier les fonctions précédentes pour obtenir une fonction `int est_abr_definition(Arbre a, long long * nb_visites)` qui valide le statut de recherche de l'arbre binaire `a`, tout en déterminant le nombre de nœuds visités arriver au résultat.

### 1.3 Application de la caractérisation d'un ABR par son parcours infixe

On rappelle qu'un arbre binaire d'entiers sans doublon est un ABR si, et seulement si, son parcours infixe visite les nœuds dans un ordre strictement croissant.

L'objectif est d'implémenter cette vérification de manière efficace, c'est-à-dire en ne parcourant l'arbre qu'une seule fois. Pour cela, on conservera tout au long des appels récursifs l'adresse du **dernier nœud visité** lors du parcours infixe afin de comparer sa valeur à celle du nœud courant.

1. Écrire la fonction `int infixe_croissant(Arbre a, Noeud ** dernier_noeud)` qui réalise un parcours infixe (potentiellement incomplet) de l'arbre `a` :
   - l'appel initial de cette fonction sera toujours réalisé avec un pointeur `*dernier_noeud` initialisé à `NULL`. Il ne sera mis à jour à l'adresse d'un nœud de l'arbre qu'une fois que le parcours infixe aura atteint le nœud le plus à gauche de l'arbre, *i.e.* sa potentielle plus petite valeur.
   - la fonction s'interrompt immédiatement et renvoie 0 si elle détecte que la valeur du nœud pointé par `*dernier_noeud` est supérieure à l'étiquette du nœud courant.
   - elle met à jour `*dernier_noeud` après la fin de la visite d'un nœud.
   - la fonction renvoie 1 si le parcours infixe a été intégralement réalisé, *i.e.* l'arbre binaire `a` possède un parcours infixe croissant et est bien un arbre binaire de recherche, 0 sinon.
2. Écrire la fonction `int est_abr_infixe(Arbre a)` qui valide le statut de recherche de l'arbre binaire `a` par la caractérisation de son parcours infixe.
3. Modifier les fonctions précédentes pour obtenir une fonction `int est_abr_infixe(Arbre a, long long * nb_visites)` qui, en plus de déterminer si l'arbre `a` est un arbre binaire de recherche, détermine aussi le nombre de nœuds visités au cours de son parcours infixe.

## 2 Comparaison des méthodes

L'objectif est maintenant de comparer expérimentalement l'efficacité des trois méthodes précédentes, en particulier d'observer comment la structure de l'arbre influence le nombre de nœuds visités par chaque méthode.

### 2.1 Génération d'arbres

Dans un premier temps, nous allons générer des arbres binaires sans doublons dont la forme est spécifique. Les fonctions suivantes seront à écrire dans un fichier `genere_arbre_binaire.c` ; son fichier `.h` associé sera aussi crée.

1. Écrire la fonction `int construit_presque_complet(Arbre * a, int ** src, int n)` qui construit dans `*a` un arbre binaire presque complet contenant exactement `n` nœuds dont les valeurs seront extraites successivement du tableau `*src` passé en paramètre dont la taille est égale à `n`.
   — L'arbre est construit selon un parcours en profondeur préfixe.
   — On utilisera un double pointeur `src` pour avancer dans le tableau de données à chaque création de nœud.
   — La fonction renverra 1 en cas de succès, et 0 en cas d'erreur d'allocation.
   — En cas d'échec, la fonction devra libérer toute la mémoire allouée lors des appels récursifs.
2. Écrire la fonction `int construit_filiforme(Arbre * a, int ** src, char ** forme, int n)` qui construit un arbre filiforme (*i.e.*, chaque nœud possède au plus un enfant) contenant `n` nœuds.
   — Les valeurs sont extraites de `*src` et la direction de l'enfant (gauche ou droite, noté `'g'` et `'d'` dans la chaine de caractères `*forme`) est déterminée par le caractère courant de `*forme`.
   — Les tableaux `*src` et `*forme` seront supposé être de taille `n`.
   — On utilisera encore des double pointeurs pour que chaque appel récursif fasse avancer la lecture dans les tableaux lors d'une création de nœud.
   — La fonction renverra 1 en cas de succès, et 0 en cas d'erreur d'allocation.
   — En cas d'échec, la fonction devra libérer toute la mémoire allouée lors des appels récursifs.
3. On rappelle que si `a` est un arbre binaire dont les étiquettes sont des entiers positifs, alors celui-ci peut être représenté par la suite d'entiers obtenue par un parcours en profondeur préfixe, un nœud étant codé par son étiquette, un sous-arbre vide étant codé par $-1$. Ce codage peut-être réutilisé pour reconstruire entièrement l'arbre a posteriori.
   Écrire la fonction `int construit_quelconque(Arbre * a, int ** codage, int n)` qui construit un arbre binaire à partir de son codage, de taille `n`, passé lors de l'appel de fonction par le pointeur sur le tableau d'entiers `*codage`.
   — L'arbre est construit selon un parcours en profondeur préfixe.
   — On utilisera à nouveau un double pointeur pour avancer dans le tableau `*codage` à chaque création de nœud.
   — La fonction renverra 1 en cas de succès, et 0 en cas d'erreur d'allocation.
   — En cas d'échec, la fonction devra libérer toute la mémoire allouée lors des appels récursifs.
4. Déduire des trois questions prédédentes les fonctions suivantes qui créent aléatoirement des arbres respectivement presque complets, filiformes et de forme quelconque, sans doublons, qui sont (selon le nom) ou non des ABR :

```
int ABR_presque_complet_alea(Arbre * a, int taille)
int non_ABR_presque_complet_alea(Arbre * a, int taille)
int ABR_filiforme_alea(Arbre * a, int taille)
int non_ABR_filiforme_alea(Arbre * a, int taille)
int ABR_quelconque_alea(Arbre * a, int taille)
int non_ABR_quelconque_alea(Arbre * a, int taille)
```

*N.B. :* Ces six fonctions doivent elles-mêmes allouer et remplir les tableaux nécessaires, avec l'aide de la fonction `rand()` ou de valeurs triées, avant d'appeler les fonctions récursives de construction.

## 2.2 Protocole expérimental

L'idée est maintenant de mettre en place un protocole expérimental permettant de confronter les complexités théoriques des trois algorithmes présentés ci-dessus avec des données expérimentales. Pour cela, dans le fichier `main.c`, vous ...

1. ... générerez aléatoirement des arbres des six types précédents (presque complet, filiforme et quelconque + ABR / non ABR) ;
2. ... exécuterez les trois méthodes de vérification sur ces arbres pour obtenir des mesures sur ces méthodes ; on rappelle que pour lisser des petites variations et obtenir des courbes robustes, chaque point de mesure peut être la moyenne arithmétique de plusieurs répétitions indépendantes ;

3. ... stockerez dans un fichier `mesures.csv` les résultats obtenus. Celui-ci aura comme première ligne : `Taille;Morphologie;Methode;Nb_visites:Temps`.

Un script Python `analyse_complexite.py` analysera le fichier `mesures.csv` en créant des courbes avec la bibliothèque `matplotlib`. Ces dernières devront mettre en évidence la complexité des algorithmes utilisés. Plusieurs échelles (linéaire / logarithmique) peuvent être utilisées pour mieux mettre en évidence des éléments saillants de discussion sur les complexités.

### 2.3 Rapport et discussions des résultats

Un rapport `prenom_X_nom_X_-_prenom_Y_nom_y.pdf`, où `prenom_X` et `prenom_Y` désignent les prénoms des membres du binômes, et `nom_X` et `nom_Y` désignent leur noms de famille, sera produit. Le rapport contiendra aussi l'intégralité des noms et prénoms des membres du binômes sur la première page.

**Ce rapport est la partie la plus importante du DM** et ne doit pas être négligé. Celui-ci devra contenir trois parties :

1. une partie succincte sur la réalisation du DM et les difficultés rencontrées ;
2. une partie explicitant précisément et **justifiant** votre protocole expérimental ; en particulier, vous expliquerez comment reproduire vos expériences ;
3. une partie analysant les résultats obtenus. Elle contiendra en premier les courbes produites, puis leur analyse détaillée. Pour chaque algorithme mis en place, vous devrez notamment :

   — justifier intégralement sa complexité théorique (éventuellement selon les types d'arbres, si cela a lieu d'être) ;
   — prouver que l'expérimentation que vous venez de faire est en cohérence avec la complexité théorique de l'algorithme ;
   — expliquer, s'il y en a, pourquoi il y a des différences entre les types d'arbres utilisés pour réaliser les mesures, ou, dans le cas contraire, expliquer pourquoi il n'y en a pas.

   Dans cette partie d'analyse, il est aussi possible de comparer les algorithmes entre eux. Par exemple, on pourra regarder l'impact de l'interruption au plus tôt du programme lorsque l'"erreur" détectée est proche de la racine dans un arbre binaire qui n'est pas de recherche, selon les méthodes.

## 3 Conditions de rendus

Ce devoir est à réaliser en binôme au sein d'un même groupe de TD. Le devoir est à rendre sur le e-learning du cours pour le dimanche 22 mars 2026, 23h59.

Le code `C` sera réalisé dans les fichiers nommé `est_ABR.c`, `genere_arbre_binaire.c`, ainsi que dans un `main.c`. Un fichier de sauvegarde `est_ABR_V1.c` sera aussi présent. Les structures et prototypes de fonctions proposées dans le sujet ne doivent pas être modifiées et seront stockés dans des `.h`. En cas de besoin, des fonctions intermédiaires peuvent être ajoutées.

Pour compiler votre fichier, vous utiliserez alors les lignes de commande suivantes (voir plus tard en Perf. C pour avoir plus d'information sur la programmation modulaire) :

```
clang est_ABR.c -c -o est_ABR.o
clang genere_arbre_binaire.c -c -o genere_arbre_binaire.o
clang main.c -c -o main.o
```

⇝ transforme les fichiers `est_ABR.c`, `genere_arbre_binaire.c`, et `main.c` en un fichier objet

```
clang est_ABR.o genere_arbre_binaire.o main.o -o main
```

⇝ crée un exécutable `main` à partir de l'ensemble de votre code.

Le code python sera réalisé dans un fichier nommé `analyse_complexite.py`.

Un fichier `.zip` y sera déposé sur le e-learning et contiendra a minima :

1. les huit fichiers de code `.c`, `.h` et `.py` que vous avez développé ;
2. le rapport nommé sous la forme `prenom_X_nom_X_-_prenom_Y_nom_y.pdf`.

L'archive sera nommé selon la nomenclature `nom1_nom2.zip` où `nom1` et `nom2` désigne les noms de famille des membres du binôme.

La correction s'effectuera en plusieurs parties :

1. le code produit sera testé de manière automatique : sur 4 points, approximativement. Il est donc particulièrement important que vous respectiez scrupuleusement l'intégralité de la nomenclature donnée dans l'énoncé ;
2. le rapport sera lu et noté par votre enseignant de TD : sur 8 points approximativement ;
3. une évaluation sous Platon sera mise en place pour évaluer votre capacité à refaire le développement que vous avez réalisé : sur 8 points approximativement.